操作系统实验报告

Lab01: UNIX Shell with History Feature

姓名：黄译辉　　学号：24281162　　日期：2026 年 10 月 13 日

一、实验目标

本实验要求设计一个 C 语言程序作为 shell 接口：接收用户命令并在单独的子进程中执行。通过实验达成以下目标：

- 深入理解操作系统 shell 的工作原理与系统调用的使用方法；
- 练习使用 fork() 创建子进程，并用 execvp() 在子进程中加载执行新程序映像；
- 掌握父进程通过 wait()/waitpid() 与子进程的运行进行协调，理解前台/后台（&）两种执行模式；
- 在此基础上实现 history 历史命令功能（history、!!、!N）及相应的错误处理。

二、Linux 进程控制方法总结

2.1 进程创建：fork()

fork() 是 Linux 创建进程的核心系统调用。它把当前进程复制一份（采用写时复制（Copy-On-Write）技术，父子进程最初共享同一物理内存页，写入时才真正复制），调用一次、返回两次：在父进程中返回子进程 PID，在子进程中返回 0，失败返回 -1。程序据此用 if-else 区分父子进程执行不同分支。

2.2 程序替换：exec 函数族

exec 函数族用磁盘上的新程序替换当前进程的代码、数据和堆栈段，PID 不变。共有 6 个变体，命名含义：l 表示参数以可变参数表逐一列出，v 表示参数装入矢量数组 char *argv[]，p 表示按 PATH 环境变量搜索可执行文件，e 表示显式传递环境变量。

函数	参数形式	是否搜索 PATH	是否带环境变量
execl	可变参数表	否	否（继承）

execlp	可变参数表	是	否（继承）
execle	可变参数表	否	是
execv	参数数组	否	否（继承）
execvp	参数数组	是	否（继承）
execve	参数数组	否	是

exec 成功时不会返回；只有失败才返回 -1。shell 中必须使用 execvp(args[0], args)：既要按命令名在 PATH 中搜索，又要传递解析好的参数数组。

2.3 进程等待：wait() 与 waitpid()

父进程调用 wait()/waitpid() 同步子进程并回收其退出状态，避免子进程变成僵尸进程。wait(&status) 等待任一子进程结束；waitpid(pid, &status, options) 可指定等待某个子进程，options 传 WNOHANG 时变为非阻塞轮询（无已结束的子进程立即返回 0）。常用宏：WIFEXITED(status) 判断是否正常退出，WEXITSTATUS(status) 取退出码。

2.4 进程终止：exit() 与 _exit()

exit(status) 正常终止进程并刷新 stdio 缓冲区，退出状态可供父进程用 wait 获取；_exit(status) 直接陷入内核不刷新缓冲区。exec 失败后子进程应立即 exit(EXIT_FAILURE)，绝不能让子进程返回继续执行 shell 主循环。

2.5 本实验中各系统调用的使用方式

系统调用/函数	作用	本实验中的使用
fork()	创建子进程	解析命令后 fork，子进程执行命令，父进程协调等待
execvp()	按 PATH 搜索并替换程序映像	子进程中 execvp(args[0], args) 执行用户命令
waitpid()	等待指定子进程 / 非阻塞回收	前台：waitpid(pid, NULL, 0) 阻塞等待；主循环开头 waitpid(-1, NULL, WNOHANG) 循环回收后台子进程，防僵尸

exit()	终止进程	exit 终止 shell；子进程 execvp 失败时 exit(EXIT_FAILURE)

三、程序设计

3.1 总体结构

程序采用经典的 shell 主循环（读入—解析—执行）结构：

```
while (should_run) {
    回收已结束的后台子进程（waitpid WNOHANG）；  /* 防僵尸 */
    printf("osh>"); fflush(stdout);             /* 显示提示符 */
    fgets 读取一行命令（最多 80 字符，超长则丢弃剩余并提示）；
    空命令 -> continue;
    !! / !N -> 从历史缓冲区取出原命令并回显（失败则报错 continue）；
    命令存入历史;                                  /* 循环缓冲区 */
    内部命令 history / exit 直接处理;
    外部命令 -> 解析 args[] -> 检查 & -> fork()
        子进程：execvp(args[0]，args)，失败则报错并 exit
        父进程：带 & 则不等待（打印子进程 pid），否则 waitpid 阻塞等待
}
```

命令分为两类：内部命令（history、exit）由父进程直接处理，无需创建子进程，且 history 自身不记入历史；外部命令通过 fork + execvp 在子进程中执行。

3.2 关键数据结构

- 参数数组：char *args[MAX_LINE/2 + 1]，存放 strtok 解析出的各 token，以 NULL 结尾，正好与 execvp 的参数约定一致（80 字符命令最多产生 40 个参数）。"&" 不计入 args。
- 历史缓冲区：char history[10][80] 循环数组 + 全局计数器 history_count。计数器即最新命令的编号（从 1 连续递增），编号 n 的命令存放于下标 (n-1) % 10，缓冲区满后自动淘汰最旧命令，因此始终保留编号 history_count-9 ~ history_count 的最近 10 条。

3.3 Part I：进程控制的关键实现

- 命令解析：strtok(line, " \t") 逐个取出 token 存入 args[]；支持 "cmd &" 与 "cmd&" 两种后台写法——末 token 恰为 "&" 时删除该 token，末字符为 & 时就地截断。
- 前台执行：父进程 waitpid(pid, NULL, 0) 点名等待本次 fork 出的子进程结束后才打印下一个提示符（不用 wait(NULL) 的原因见第五节问题 1）；
- 后台执行：检测到 & 时父进程跳过等待，打印 [后台] pid = xxx 后立即回到主循环，父子进程并发运行；
- 失败处理：子进程中 execvp 返回说明加载失败，打印错误信息后 exit(EXIT_FAILURE)，避免子进程继续运行 shell 循环。

3.4 Part II：history 功能的关键实现

```
/* 编号 n 的命令存放在 (n-1)%10 号槽位；有效编号范围 [oldest, count] */
int oldest = history_count > HISTORY_NUM ? history_count - HISTORY_NUM + 1 :
1;
history 打印: for (i = history_count; i >= oldest; i--)
                  printf("%d %s\n", i, history[(i - 1) % HISTORY_NUM]);
!N 查询:     n 在 [oldest, history_count] 内才取出 history[(n-1)%10],
             否则打印 "No such command in history."
```

- history：从新到旧遍历打印 "编号 命令"，最多 10 条；history 命令本身不记入历史；
- !!：历史为空打印 "No commands in history."，否则取出最近一条；
- !N：! 后跟纯数字时按编号取出；N 越界（不在保留窗口内）打印 "No such command in history."；
- 回显与入历史：由 !!/!N 取出的命令先回显，再作为一条新命令加入历史（编号继续递增），与题目要求一致。

3.5 错误处理设计

- 命令超过 79 字符：检测到 fgets 未读到换行符时清空 stdin 缓冲区剩余内容并提示，防止残余字符被误当作下一条命令；
- 僵尸进程：主循环每轮开头用 while(waitpid(-1, NULL, WNOHANG) > 0) 回收已结束的后台子进程；
- 命令不存在：execvp 失败后子进程打印 "osh: xxx: ..." 并以失败状态退出；
- 历史引用失败（!! 无历史、!N 越界）时打印指定错误信息，且不把失败的引用本身存入历史。

四、实验结果与分析

实验环境：Windows 11 + WSL2（Ubuntu），gcc 编译，编译选项 -Wall -Wextra -O2 -std=gnu11，无任何警告。以下截图均为 2026 年 10 月 13 日本人亲手操作截取。

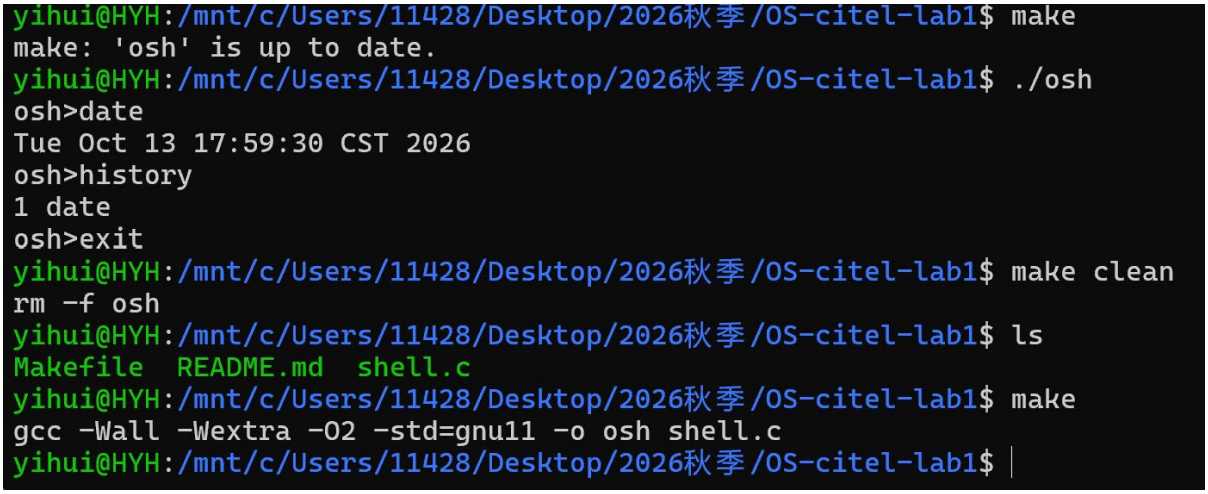


图 1 make 编译流程与 'osh' is up to date 现象

图 1 分三段：首次 make 时因 osh 比 shell.c 新，make 判定 'osh' is up to date 跳过编译（详见第五节问题 5）；make clean 删除 osh 后 ls 确认只剩源文件；再次 make 才真正执行 gcc -Wall -Wextra -O2 -std=gnu11 -o osh shell.c，全程无警告。

4.1 基本命令执行与 fork 的直观证据

```
yihui@HYH:/mnt/c/Users/11428/Desktop/2026秋季/OS-citel-lab1$ gcc -Wall -o osh shell.c
yihui@HYH:/mnt/c/Users/11428/Desktop/2026秋季/OS-citel-lab1$ ./osh
osh>date
Tue Oct 13 20:07:29 CST 2026
osh>ps -el
F S   UID     PID    PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
4 S     0       1       0  0  80   0 -  6063 -      ?        00:00:00 systemd
5 S     0       2       1  0  80   0 -   780 -      ?        00:00:00 init-systemd(Ub
0 S     0       7       2  0  80   0 -   780 -      ?        00:00:00 init
4 S     0      47       1  0  79  -1 - 12596 -      ?        00:00:00 systemd-journal
4 S   989      81       1  0  80   0 -  5603 -      ?        00:00:00 systemd-resolve
4 S     0      87       1  0  80   0 -  8888 -      ?        00:00:00 systemd-udevd
4 S     0     205       1  0  80   0 -   720 -      ?        00:00:00 chronyd-starter
4 S     0     206       1  0  80   0 -  1116 -      ?        00:00:00 cron
4 S   995     207       1  0  80   0 -  2196 -      ?        00:00:00 dbus-daemon
4 S     0     211       1  0  80   0 - 11024 -      ?        00:00:00 networkd-dispat
4 S     0     215       1  0  80   0 -  4605 -      ?        00:00:00 systemd-logind
4 S     0     229       1  0  80   0 -  1382 -      hvc0     00:00:00 agetty
4 S     0     238       1  0  80   0 -  1371 -      tty1     00:00:00 agetty
4 S   100     247       1  0  80   0 - 55133 -      ?        00:00:00 rsyslogd
4 S   988     257     205  0  80   0 -  5124 -      ?        00:00:00 chronyd
4 S     0     263       1  0  80   0 - 30865 -      ?        00:00:00 unattended-upgr
1 S   988     269     257  0  80   0 -  3020 -      ?        00:00:00 chronyd
4 S     0     376       2  0  80   0 -  2157 -      ?        00:00:00 login
4 S  1000     419       1  0  80   0 -  5583 -      ?        00:00:00 systemd
5 S  1000     423     419  0  80   0 -  5726 -      ?        00:00:00 (sd-pam)
4 S  1000     451     376  0  80   0 -  1532 do_sel pts/1    00:00:00 bash
5 S     0     515       2  0  80   0 -   782 -      ?        00:00:00 SessionLeader
5 S     0     516     515  0  80   0 -   786 -      ?        00:00:00 Relay(518)
4 S  1000     518     516  0  80   0 -  1564 do_wai pts/0    00:00:00 bash
4 S     0    1081       1  0  80   0 - 466506 -     ?        00:00:00 wsl-pro-service
0 S  1000    1127     518  0  80   0 -   691 do_wai pts/0    00:00:00 osh
0 R  1000    1129    1127  0  80   0 -  1810 -      pts/0    00:00:00 ps
osh>
```

图 2 前台执行 date 与 ps -el，ps 的 PPID 指向 osh

date 正常输出当前时间；ps -el 的进程表中，osh 自身 PID 为 1127，而末行 ps 命令的 PID 为 1129、PPID（父进程号）恰为 1127——ps 进程正是 osh 通过 fork() 创建的子进程，直观证明了"shell 创建子进程执行命令"的工作方式。ps 结束后父进程才打印下一个 osh> 提示符，体现 waitpid 的同步作用。

4.2 history 列表与 !! / !N 引用

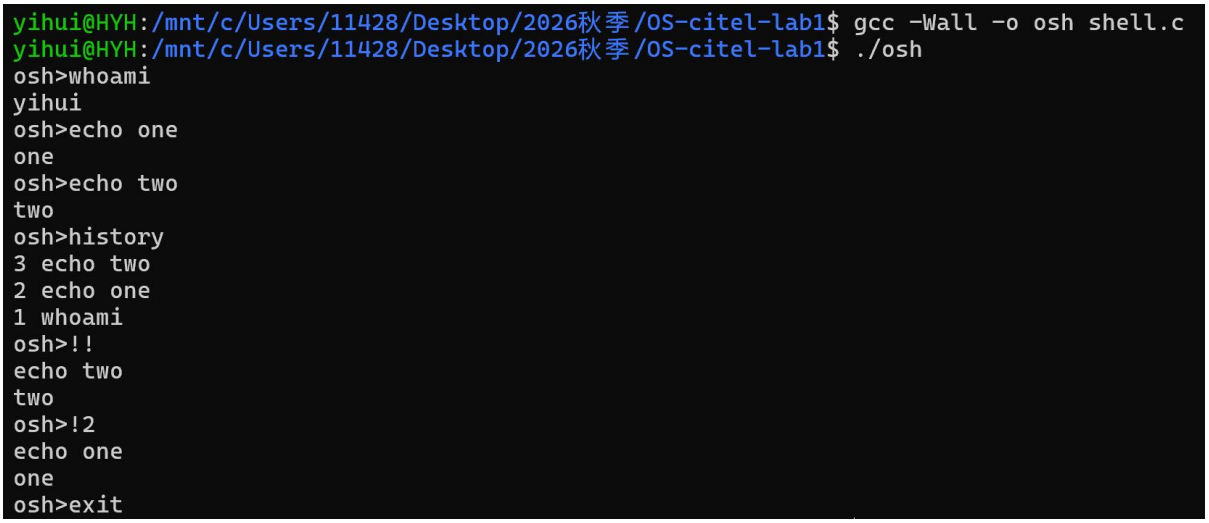


图 3 history 从新到旧列出，!! 与 !2 先回显再执行

依次输入 whoami、echo one、echo two 后，history 输出 3 echo two、2 echo one、1 whoami：编号从新到旧排列，与题目示例格式一致，且 history 命令本身不在列表中（不入史）。!! 回显 echo two 并执行（输出 two），!2 回显 echo one 并执行（输出 one），被引用执行的命令作为新命令入史，因此继续输入命令后编号继续递增。

4.3 后台运行（&）与父子进程并发

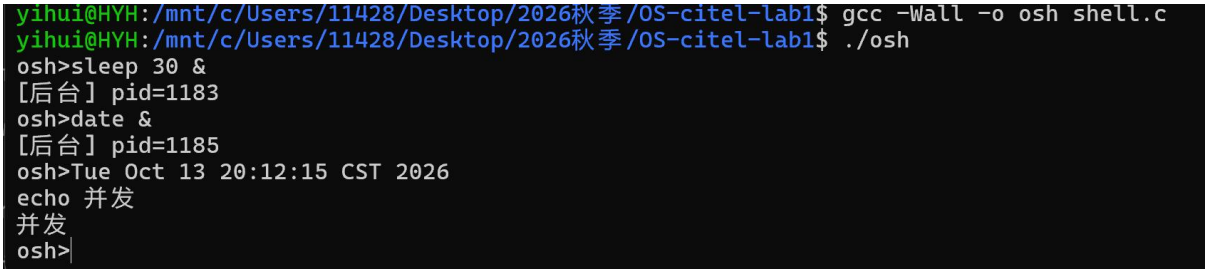
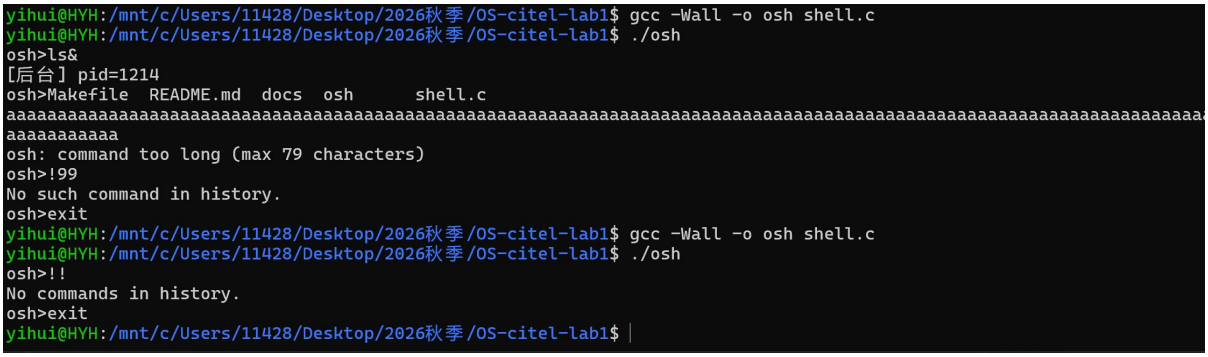


图 4 sleep 30 & 与 date & 立即返回提示符，父子进程并发

输入 sleep 30 & 后 shell 立即打印 [后台] pid=1183 并显示下一个提示符；紧接着的 date & 同样立即返回（pid=1185）。date 的输出 Tue Oct 13 20:12:15 CST 2026 在提示符之后异步插入屏幕——证明父进程没有阻塞，父子进程并发运行。数秒后 sleep 结束，由主循环开头的 waitpid(WNOHANG) 自动回收，不产生僵尸进程。

4.4 边界情况测试


yihui@HYH:/mnt/c/Users/11428/Desktop/2026秋季/OS-citel-lab1$ gcc -Wall -o osh shell.c
yihui@HYH:/mnt/c/Users/11428/Desktop/2026秋季/OS-citel-lab1$ ./osh
osh>ls&
[后台] pid=1214
osh>Makefile README.md docs osh shell.c
aaaa

```
yihui@HYH:/mnt/c/Users/11428/Desktop/2026秋季/OS-citel-lab1$ ./osh
osh>echo c1
c1
osh>echo c2
c2
osh>echo c3
c3
osh>echo c4
c4
osh>echo c5
c5
osh>echo c6
c6
osh>echo c7
c7
osh>echo c8
c8
osh>echo c9
c9
osh>echo c10
c10
osh>echo c11
c11
osh>echo c12
c12
osh>history
10 echo c10
9 echo c9
8 echo c8
7 echo c7
6 echo c6
5 echo c5
4 echo c4
3 echo c3
2 echo c12
1 echo c11
osh>!1
echo c11
c11
```

图 6 修复前：编号仍打 1~10，内容错位，!1 执行了被顶掉的命令

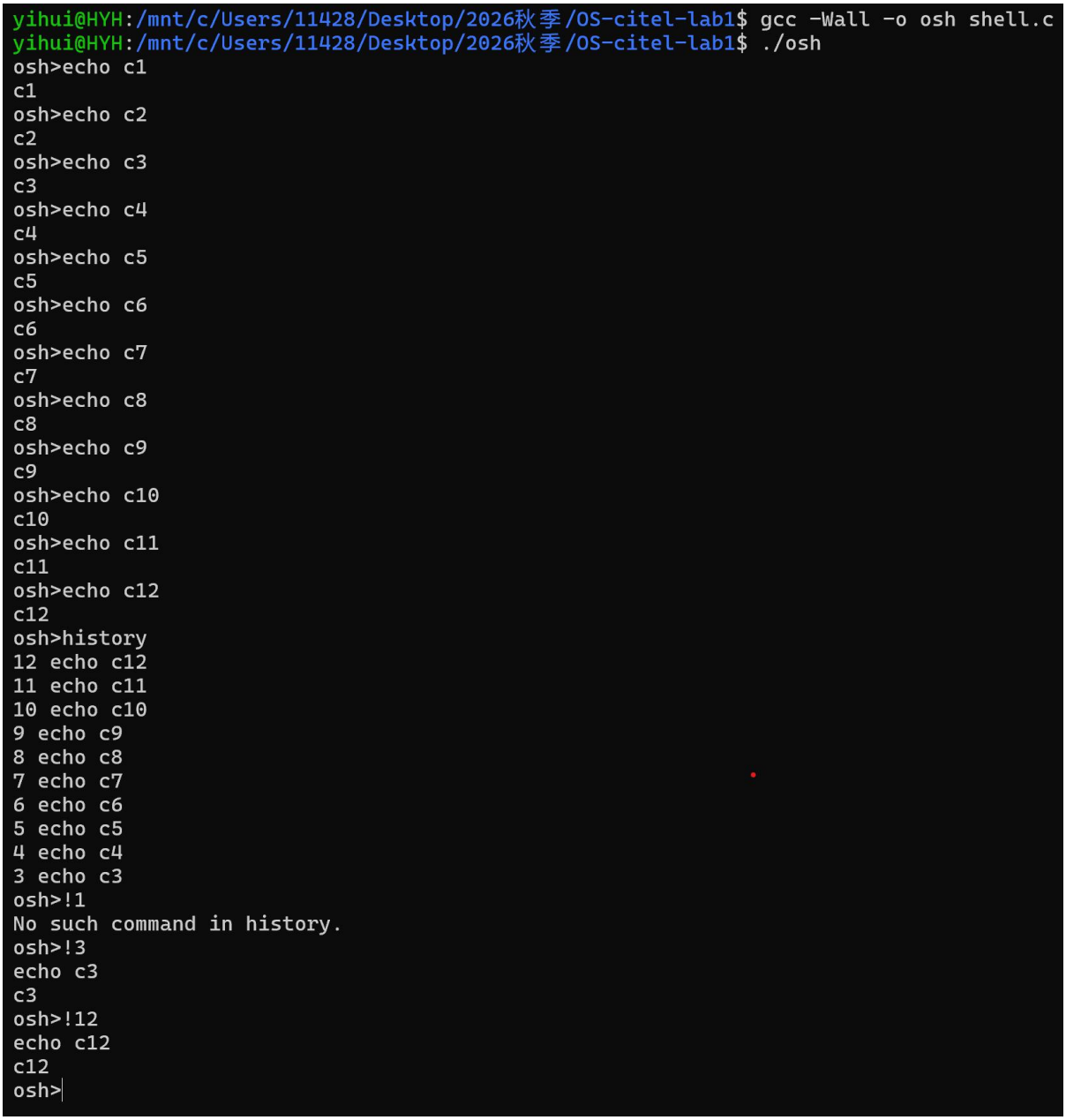


图 7 修复后：编号 12~3 与内容一一对齐，越界编号正确报错

连续执行 echo c1 ~ echo c12 共 12 条命令。修复前（图 6）：history 仍从 1 打到 10，其中 2 echo c12、1 echo c11 明显错位；更严重的是 !1 不报错，反而把窗口内 1 号槽位现存的 echo c11 执行了。修复后（图 7）：history 显示编号 12~3 的最近 10 条，编号连续递增且与命令一一对齐，最旧的 c1、c2 已被自动淘汰，与题目"输入 35 条命令时显示 26~35"的要求一致；!1 正确报 No such command in history.，!3、!12 均能准确取出对应命令执行。

五、遇到的问题及解决方法

1. **后台子进程变僵尸，且 wait(NULL) 会"错收"孩子：**初版实现后台命令后测试发现，ps 中残留一条状态为 Z（<defunct>）的 echo 进程——前台命令反而成了僵尸。分析时序后确认两个缺陷叠加：后台的 date 先于前台 echo 结束，而前台父进程用的 wait(NULL) 语义是"等任意一个孩子"，把 date 的退出状态收走后轮到 echo 就无人回收；后台死者本身也没有回收逻辑。解决：前台改用 waitpid(pid, NULL, 0) 点名等待本次子进程，主循环每轮开头用 while(waitpid(-1, NULL, WNOHANG) > 0) 非阻塞清扫已结束的后台子进程。复测同一组命令，ps 中不再出现 Z 状态条目。

2. **命令超过 10 条后 history 编号错乱，!N 会执行被顶掉的命令：**敲满 12 条后 history 仍打印编号 1~10，其中 2 显示 echo c12、1 显示 echo c11；且 !1 不报错，把 1 号槽位现存的 echo c11 执行了。与 AI 讨论定位：打印与查询都默认"格子 i-1 住着第 i 条命令"，循环缓冲区转一圈覆盖后该假设不成立。解决：先算窗口下限 oldest（超过 10 条时为 history_count-9，否则为 1），history 从 history_count 倒序打到 oldest、槽位取 (i-1)%10；!N 只接受 [oldest, history_count] 区间内的编号。修复前后对比见图 6、图 7。

3. **strtok 会破坏原字符串，记历史的顺序不能反：**strtok 解析时把分隔符就地改写成 '\0'，若先用 strtok 切词再存历史，"ls -l" 进历史就只剩 "ls"。解决：把 strcpy 入史的代码放在 strtok 之前，先保存完整命令行再解析。

4. **超长命令的残留输入：**fgets 只读 79 字符，超长行的剩余字符留在 stdin 缓冲区，会被下一轮 fgets 误当作新命令。解决：检测缓冲区中无换行符即判定超长，循环 getchar() 丢弃剩余输入并报错提示。实测输入 100 余个字符后 shell 正常继续，历史未被污染（用 !! 验证找回的是上一条正常命令）。

5. **make 显示 'osh' is up to date 不编译：**首次使用 Makefile 时 make 没有执行 gcc。原因是 make 按文件修改时间判断目标是否过期：此前手动 gcc 编译出的 osh 比 shell.c 新，make 认为无需重建。解决：make clean 删除产物后再 make，观察到 gcc 命令真正执行。这加深了对 make 增量构建机制的理解。

6. **!!/!N 的历史记录语义：**按题目要求"被执行的命令应作为下一条命令放入历史"，实现时先解析展开、回显后再把展开后的命令（而非 "!!" 字符串本身）存入历史，因此连敲两次 !! 历史里会出现两条相同命令；解析失败（无历史/越界）则直接进入下一轮循环，不写入历史，保证历史内容始终可执行。

六、参考资料

- Abraham Silberschatz 等. Operating System Concepts (10th Edition), Chapter 3: Processes.
- W. Richard Stevens 等. UNIX 环境高级编程（第 3 版）第 8 章：进程控制.
- Linux 手册页：man 2 fork、man 3 execvp、man 2 waitpid、man 3 exit.

七、建议与意见

- 实验指导中未明确 history 命令本身、以及被 !!/!N 引用执行的命令是否计入历史，建议在题面中给出明确规定或示例，减少理解的二义性；
- 建议提供一组自动化测试输入用例（如超长命令、空命令、连续 !!、超过 10 条后的越界 !N 等），便于自测与助教验收；
- 本次实验最大的收获是亲眼观察到教科书概念在真实系统中的形态：用 ps 抓到过僵尸进程、复现过历史编号错乱再亲手修复，还顺带弄清了 make 的增量构建行为。"读取—解析—fork—exec—wait" 这个循环从此不再是抽象流程图。

八、AI 使用说明

按照课程"允许使用 AI 大模型，但必须明确说明各自所做工作"的要求，如实申报本实验的人机分工：

AI（Trae 智能体）所做：讲解 fork/execvp/waitpid、循环缓冲区等概念；按实验要求撰写全部 C 代码、Makefile 与文档初稿；起草各次 git 提交的说明文字。

黄译辉（24281162）所做：逐行读懂全部代码并当场提问讨论；在 WSL Ubuntu 中亲手完成每一次编译与交互测试；亲手触发并发现僵尸进程、history 编号错乱两个真实 bug 并与 AI 讨论定位原因；全部实验截图由本人操作截取；逐字审阅并修改每一次提交说明、聊天记录与本报告，定稿后方才推送。

支撑材料：仓库 docs/ 目录收录与 AI 的完整聊天记录整理（三份），git 提交历史完整记录每一步推进、踩坑与修复过程（git log），多次提交说明中注明"与 AI 讨论分析的结论"。